EduStack Masterclass

Your Ultimate Computer Science & Engineering Interview Preparation Hub

VOLUME 2 of 4 — Authentication, Security, Cryptography & OAuth 2.0

Auth & Security: From Zero to FAANG Interview Ready

bcrypt | JWT | Cookies | Google OAuth | OTP | HMAC-SHA256 | XSS | CSRF | NoSQL Injection | OWASP | 40 Deep Q&As

Project:	EduStack - CS Student Resource Hub & AI Tutor Platform
This Volume:	Auth, Security, Cryptography, JWT, bcrypt, OAuth 2.0, OTP, OWASP, 40 Q&As
Volume 1:	JS Engine, Node.js Event Loop, Express Pipeline, REST API
Volume 3:	MongoDB, Mongoose, Indexing, Caching, Cloudinary, Sessions
Volume 4:	System Design, OS, DSA Patterns, Microservices, FAANG Scenarios
Auth Stack:	bcryptjs (12 rounds) + jsonwebtoken + Passport.js + express-session + Nodemailer
Payment Security:	Razorpay HMAC-SHA256 + crypto.timingSafeEqual (fraud prevention)
Cookie Security:	httpOnly + Secure + SameSite - XSS and CSRF prevention
Target Roles:	SDE I/II/III - Backend, Security Engineer, Full-Stack

Table of Contents — Volume 2: Auth & Security

SECTION 1

Cryptography Foundations

Hashing vs encryption, symmetric vs asymmetric, salt, pepper - from first principles

1.1 Hashing vs Encryption - A Fundamental Distinction

Hashing and encryption are often confused but are fundamentally different operations. Understanding this difference is critical for designing secure authentication systems.

Property	Hashing (One-Way)	Encryption (Two-Way)
Reversibility	IRREVERSIBLE - cannot get original from hash	REVERSIBLE - decryption produces original plaintext
Purpose	Verify data integrity without storing original	Store/transmit data that must be recovered
Key required?	No key (deterministic function)	Yes - encryption key + decryption key
Examples	SHA-256, SHA-512, bcrypt, MD5 (broken)	AES-256 (symmetric), RSA (asymmetric)
Used for	Passwords, checksums, digital signatures, JWTs	HTTPS/TLS, file encryption, secure key exchange
EduStack usage	bcrypt for passwords, SHA-256 for Razorpay HMAC	TLS for all HTTPS connections (handled by Render.com)

1.2 Symmetric vs Asymmetric Encryption

- **Symmetric encryption**: Same key encrypts and decrypts. Fast. Examples: AES-128, AES-256, ChaCha20. Problem: How do you securely share the key? Used in: TLS bulk data transfer after key exchange.
- **Asymmetric encryption**: Public key encrypts, private key decrypts (or private key signs, public key verifies). Slow. Examples: RSA-2048, RSA-4096, ECC (Elliptic Curve). Solves key distribution problem. Used in: TLS handshake key exchange, digital signatures, JWT RS256.
- **TLS (HTTPS) uses BOTH**: Asymmetric RSA/ECDH for key exchange (slow, but only done once per connection), then symmetric AES for bulk data transfer (fast, uses the exchanged shared key).
- **EduStack JWT uses HS256 (HMAC-SHA256) - symmetric**: the same JWT_SECRET is used to sign and verify. For public-facing APIs, RS256 (RSA) is preferred so verification can be done without the private key.

1.3 Salt, Pepper & Rainbow Tables

Why not just SHA-256 passwords? Because SHA-256 is deterministic: SHA256("password123") always produces the same hash. An attacker can precompute a "rainbow table" - a giant lookup table of hash->plaintext mappings - and instantly reverse common passwords.

Defense	What It Is	How bcrypt Uses It	Who Knows It
Salt	Random unique value added to each password before hashing	bcrypt generates a random salt per-user and embeds it IN the hash string (after the cost prefix)	Stored in DB (not secret - its uniqueness prevents rainbow tables)
Pepper	Secret value added to all passwords, stored in env (not DB)	NOT used in EduStack (optional additional layer, adds complexity)	Known only to server, never stored in DB
Cost factor	Number of bcrypt rounds (2^N iterations)	EduStack uses 12 rounds = 4096 iterations per hash, ~300ms on modern CPU	Embedded in hash string as 12 prefix

KEY CONCEPT: bcrypt output format: $2b$12$N9qo8uLOickgx2ZMRZoMyuDkspedmjklr9.kHc36M3KDu30fxqhBP. Breaking it down: $2b$ = bcrypt version, 12 = cost factor (12 rounds), next 22 chars = base64-encoded salt, remaining = the actual hash. bcrypt.compare() extracts the salt and cost from this string automatically.

SECTION 2

bcrypt Deep Dive

Blowfish cipher, 12 cost rounds, why bcrypt beats SHA-256 for passwords

2.1 Why bcrypt for Passwords?

bcrypt is a deliberately SLOW hashing algorithm designed specifically for passwords. Regular hash functions (SHA-256, MD5) are designed to be as FAST as possible - SHA-256 can hash 1 billion passwords per second on a GPU. bcrypt at cost factor 12 does ~100 hashes per second on the same GPU. This 10-million-fold slowdown makes brute force attacks economically infeasible.

JavaScript / Node.js

```
// bcrypt implementation in EduStack  -  authController.js
const bcrypt = require("bcryptjs");
const SALT_ROUNDS = 12; // 2^12 = 4096 iterations per hash

// === DURING REGISTRATION ===
exports.register = asyncHandler(async (req, res) => {
  const { password } = req.body;
  // bcrypt.hash() does 3 things:
  // 1. Generates a cryptographically random 128-bit salt
  // 2. Runs Blowfish key expansion 2^12 = 4096 times
  // 3. Returns the combined hash string with embedded salt + cost
  const hashedPassword = await bcrypt.hash(password, SALT_ROUNDS);
  await User.create({ password: hashedPassword, ... });
});

// === DURING LOGIN (instance method on User model) ===
userSchema.methods.comparePassword = async function(candidatePassword) {
  // bcrypt.compare() extracts the salt from this.password (stored hash),
  // hashes candidatePassword with the SAME salt + cost, then compares.
  // Returns true if match, false if not.
  return bcrypt.compare(candidatePassword, this.password);
};

// === PRE-SAVE HOOK (safety net in models/user.js) ===
userSchema.pre("save", async function(next) {
  if (!this.isModified("password") || !this.password) return next();
  // Guard: if already a bcrypt hash ($2a$/$2b$), do not double-hash
  if (!this.password.startsWith("$2a$") && !this.password.startsWith("$2b$")) {
    const salt = await bcrypt.genSalt(12);
    this.password = await bcrypt.hash(this.password, salt);
  }
  next();
});
```

FAANG TIP: Interview Q: "Why does EduStack use 12 bcrypt rounds instead of 10 or 14?" 10 rounds (~100ms) is sufficient but 12 (~300ms) is the current production recommendation balancing security and UX. 14 rounds (>1 second) would make login noticeably slow. The bcrypt cost factor should be increased as hardware improves to maintain the target hash time of 100-300ms.

SECTION 3

JWT - Structure, Signing & Verification

Header.Payload.Signature, HS256, minimal payload, why role is NOT stored in token

3.1 JWT Structure - Decoded

A JSON Web Token (JWT) is a compact, URL-safe way to securely transmit claims between parties. It consists of three Base64URL-encoded parts separated by dots: Header.Payload.Signature. The signature makes it tamper-evident - any modification to the header or payload invalidates the signature.

JavaScript / Node.js

```
// JWT HEADER (Base64URL decoded):
{
  "alg": "HS256",  // Algorithm: HMAC-SHA256 (symmetric)
  "typ": "JWT"
}

// JWT PAYLOAD (Base64URL decoded)  -  EduStack keeps this MINIMAL:
{
  "id": "64f7a2b3c8e1234567890abc",  // MongoDB ObjectId of the user
  "iat": 1724464800,                  // Issued At (Unix timestamp)
  "exp": 1725069600                   // Expiry (iat + 7 days)
}
// NOTE: role and email are intentionally NOT in the payload.
// Reason: If an admin revokes a user, the token still has the old role.
// EduStack re-fetches user from DB on every request (isAuth) so revocations
// take effect immediately, not just after token expiry.

// JWT SIGNATURE:
// HMAC-SHA256(base64url(header) + "." + base64url(payload), JWT_SECRET)

// utils/generateToken.js
const generateToken = (userId) => {
  return jwt.sign(
    { id: userId },                          // Minimal payload
    process.env.JWT_SECRET,                  // Symmetric secret key
    { expiresIn: process.env.JWT_EXPIRES_IN || "7d" } // 7 day default
  );
};

// jwt.verify() in isAuth middleware:
const decoded = jwt.verify(token, process.env.JWT_SECRET);
// decoded = { id: "...", iat: ..., exp: ... }
// Throws JsonWebTokenError if tampered, TokenExpiredError if expired
```

HS256 vs RS256 - When to Use Which

Property	HS256 (HMAC-SHA256)	RS256 (RSA + SHA256)
Type	Symmetric - same secret for sign and verify	Asymmetric - private key signs, public key verifies
Key Management	Single secret shared between signer and verifier	Private key secured on auth server, public key distributed freely
Performance	Fast - HMAC is a single-pass operation	Slower - RSA involves modular exponentiation
Best For	Single-server or trusted microservices that share secret	Multi-service architectures where services only need to VERIFY (not sign)
EduStack	Uses HS256 - single Node.js server signs and verifies	Would use RS256 if auth server was separate from resource servers

SECTION 4

Cookie Security - httpOnly, Secure, SameSite

How EduStack prevents XSS token theft and CSRF attacks via cookie flags

4.1 The Three Critical Cookie Flags

EduStack stores the JWT in an httpOnly cookie named "edustack_token". This is the recommended approach for web applications - it is significantly more secure than storing JWTs in localStorage, which is vulnerable to XSS attacks.

Flag	What It Does	Without It	EduStack Value
httpOnly	Prevents JavaScript from reading the cookie via document.cookie	An XSS attack can steal the token with: document.cookie (trivial)	true - always set
Secure	Cookie is only sent over HTTPS connections	Token sent in plaintext over HTTP - intercepted by man-in-the-middle	true in production, false in local dev (no HTTPS locally)
SameSite	Controls when cookie is sent in cross-origin requests	Without SameSite=strict, an attacker's site can trigger requests to EduStack API using victim's cookie	strict in dev, none in production (required for OAuth redirect)
MaxAge	Time in milliseconds before cookie expires	Session cookie - deleted when browser closes (poor UX)	7 days (604800000 ms)

JavaScript / Node.js

```
// utils/generateToken.js  -  How EduStack sets the secure cookie
const attachCookieToken = (res, userId) => {
  const token = generateToken(userId);
  const IS_PRODUCTION = process.env.NODE_ENV === "production";

  res.cookie("edustack_token", token, {
    httpOnly: true,       // CRITICAL: JS cannot read this cookie
    secure: IS_PRODUCTION, // HTTPS-only in production
    sameSite: IS_PRODUCTION ? "none" : "strict",
    // Production uses "none" because Google OAuth redirect goes from
    // Google (different origin) back to our server, and the cookie
    // must be sent in that cross-origin redirect.
    // "none" REQUIRES secure:true (HTTPS)
    maxAge: 7 * 24 * 60 * 60 * 1000, // 7 days in milliseconds
  });

  return token; // Also returned in body for API clients that prefer headers
};

// LOGOUT: Clear the cookie by setting maxAge to 0
exports.logout = asyncHandler(async (req, res) => {
  res.cookie("edustack_token", "", {
    httpOnly: true,
    secure: IS_PRODUCTION,
    sameSite: "strict",
    maxAge: 0, // Immediately expire  -  effectively deletes the cookie
  });
  return sendSuccess(res, "Logged out successfully.");
});
```

SECURITY WARNING: NEVER store JWTs in localStorage. localStorage is accessible via document.cookie/ window.localStorage from any JavaScript on the page. A single XSS vulnerability (e.g., injecting a script via unsanitized user input) can steal ALL tokens from localStorage. httpOnly cookies are immune to this attack because JavaScript cannot read them at all.

SECTION 5

EduStack Full Authentication Flow

Registration -> OTP -> verify -> JWT cookie; Login -> comparePassword -> JWT

5.1 Registration Flow - Step by Step

FLOW: EduStack User Registration Flow

Step 1: POST /api/auth/register - Client sends { firstName, lastName, email, password, avatar? }

↓

Step 2: Normalize email (lowercase + trim). Check for existing user (User.findOne({ email })) - 409 if duplicate

↓

Step 3: Hash password with bcrypt.hash(password, 12) - ~300ms, runs in libuv thread pool

↓

Step 4: Handle optional avatar: multer reads file into Buffer, bufferToBase64Uri() converts, Cloudinary uploads

↓

Step 5: User.create({ firstName, lastName, email, password: hashedPassword, avatar, role }) - saves to MongoDB

↓

Step 6: otpService.saveAndSendOtp(email) - generates 6-digit OTP, upserts to OTP collection, sends via Nodemailer

↓

Step 7: Return 201 Created with { email } - user cannot log in until OTP verified (isVerified: false)

5.2 OTP Verification Flow

FLOW: OTP Verify -> Account Activation -> JWT Issue

Step 1: POST /api/auth/verify-otp - Client sends { email, otp }

↓

Step 2: otpService.verifyOtp(email, otp) - finds OTP document in MongoDB, checks code match and expiry

↓

Step 3: User.findOneAndUpdate({ email }, { isVerified: true }, { new: true }) - activates account

↓

Step 4: mailService.sendWelcomeEmail() - fire-and-forget (no await), failure does NOT block response

↓

Step 5: attachCookieToken(res, user._id) - generates JWT, sets httpOnly cookie, returns token in body

↓

Step 6: Return 200 with { token, user: { id, firstName, lastName, email, role, avatar } }

5.3 Login Flow - The Critical password Selection

JavaScript / Node.js

```
// authController.js  -  Login: The most security-critical endpoint
exports.login = asyncHandler(async (req, res) => {
  const { email, password } = req.body;
  const normalizedEmail = email.toLowerCase().trim();

  // CRITICAL: .select("+password") is required because the User schema
  // has { password: { type: String, select: false } }
  // Without +password, user.password is undefined and comparePassword() fails.
  const user = await User.findOne({ email: normalizedEmail }).select("+password");

  if (!user) {
    // Deliberately vague message  -  do NOT say "email not found"
    // Specific messages enable email enumeration attacks.
    return sendError(res, "Invalid email or password.", 401);
  }

  // Block unverified accounts and auto-resend OTP for better UX
  if (!user.isVerified) {
    await otpService.saveAndSendOtp(normalizedEmail); // Resend OTP
    return sendError(res, "Account not verified. A new OTP has been sent.", 403);
  }

  // bcrypt.compare() hashes the candidate password with the same salt
  // that is embedded in user.password, then compares.
  const isMatch = await user.comparePassword(password);
  if (!isMatch) {
    return sendError(res, "Invalid email or password.", 401); // Same message!
  }

  const token = attachCookieToken(res, user._id); // JWT cookie set here
  user.password = undefined; // Remove from response object

  return sendSuccess(res, "Logged in successfully.", { token, user: {...} });
});
```

FAANG TIP: Interview Q: "Why does EduStack return the same error message for wrong email AND wrong password?" This prevents email enumeration attacks. If the server said "Email not found" when email is wrong and "Wrong password" when email exists, an attacker could enumerate which emails are registered. Always return the same message for authentication failures.

SECTION 6

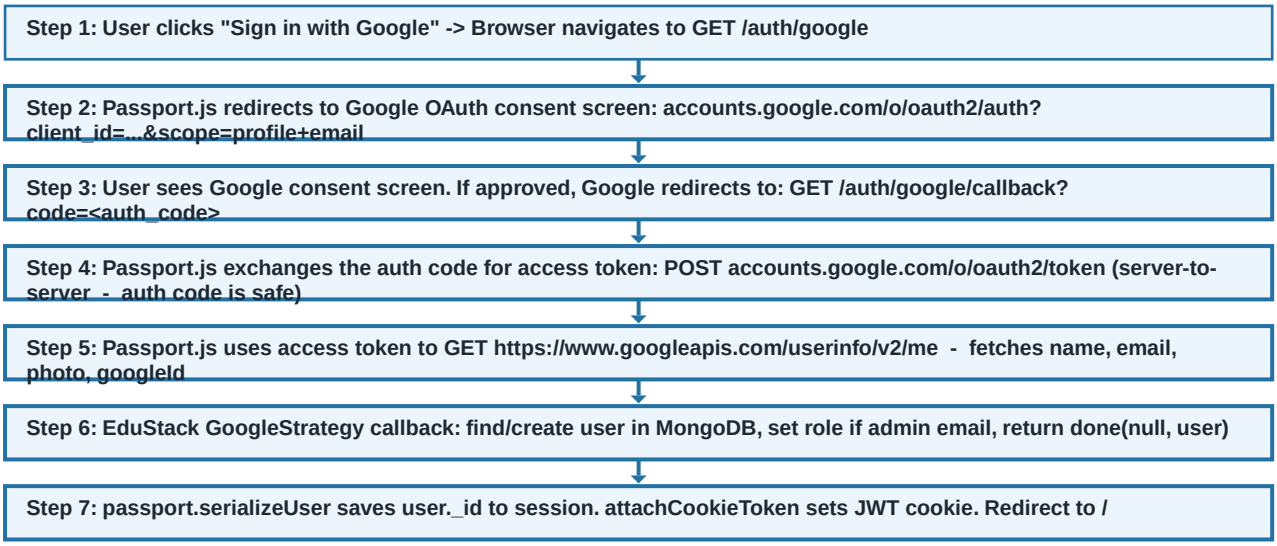
Google OAuth 2.0 Deep Dive

Authorization Code Grant flow, Passport.js, session + JWT hybrid

6.1 OAuth 2.0 - Why It Exists

OAuth 2.0 is an authorization framework that allows users to grant third-party applications limited access to their account on another service, WITHOUT sharing their password. "Sign in with Google" uses OAuth - EduStack never sees your Google password. It only receives a profile (name, email, photo) after you grant permission.

FLOW: Google OAuth 2.0 Authorization Code Grant Flow



JavaScript / Node.js

```
// app.js  -  Google OAuth Strategy Configuration
passport.use(new GoogleStrategy({
    clientID:     process.env.GOOGLE_CLIENT_ID,
    clientSecret: process.env.GOOGLE_CLIENT_SECRET,
    callbackURL:  getGoogleCallbackURL(), // Handles localhost vs Render.com
  },
  async (accessToken, refreshToken, profile, done) => {
    try {
      const userEmail = profile.emails[0].value.toLowerCase().trim();
      const ADMIN_EMAILS = (process.env.ADMIN_EMAILS || "").split(",")...
      const isAdmin = ADMIN_EMAILS.includes(userEmail);

      // 1. Find by googleId (returning Google user)
      let user = await User.findOne({ googleId: profile.id });
      if (user) { /* update role if admin, return */ return done(null, user); }

      // 2. Find by email (local account, now linking Google)
      const emailUser = await User.findOne({ email: userEmail });
      if (emailUser) {
        emailUser.googleId = profile.id; // Link Google ID to existing account
        await emailUser.save();
        return done(null, emailUser);
      }

      // 3. Create new Google user (first-time Google login)
      user = await User.create({
        firstName: profile.name.givenName || "User",
        lastName:  profile.name.familyName || "",
        email:     userEmail,
        googleId:  profile.id,
        role:      isAdmin ? "admin" : "user",
        avatar:    profile.photos?.[0]?.value || "default-avatar.png",
        isVerified: true, // Google accounts are pre-verified
      });
      return done(null, user);
    } catch (err) { return done(err, null); }
```

JavaScript / Node.js

```
  }
));
```

Why EduStack Uses BOTH Sessions AND JWTs for OAuth

- **Google OAuth REQUIRES session state**: After the consent screen, Google redirects to /auth/google/callback with an authorization code. Passport.js needs session state to know which user initiated this OAuth flow and to store temporary OAuth data between redirects.
- connect-mongodb-session stores session data in MongoDB "sessions" collection. Session ID is stored in a cookie (not httpOnly by default). Passport serializes user._id to session on login.
- **After OAuth callback**: EduStack calls attachCookieToken(res, user._id) to also set the JWT httpOnly cookie. This gives the frontend a JWT for subsequent API calls (which use Bearer token or cookie JWT auth, not session auth).
- **Hybrid design**: Session (stateful) for OAuth flow. JWT (stateless) for all API requests. Best of both worlds.

SECTION 7

OTP Service & Email Security

Random OTP generation, MongoDB TTL, upsert, fire-and-forget email pattern

7.1 OTP Implementation - Security Analysis

JavaScript / Node.js

```
// services/otpService.js  -  Simplified implementation
const OTP = require("../models/otp");
const mailService = require("./mailService");
const crypto = require("crypto");

// Generate cryptographically secure 6-digit OTP
const generateOtp = () => {
  // crypto.randomInt(min, max) uses OS CSPRNG (Cryptographically Secure PRNG)
  // Unlike Math.random() which is NOT cryptographically secure
  return String(crypto.randomInt(100000, 999999));
};

exports.saveAndSendOtp = async (email) => {
  const code = generateOtp();
  const expiresAt = new Date(Date.now() + 10 * 60 * 1000); // 10 minutes

  // upsertOne: Updates if exists, creates if not.
  // { upsert: true } prevents duplicate OTP documents per email.
  await OTP.findOneAndUpdate(
    { email },                          // find by email
    { email, code, expiresAt },         // update with new code
    { upsert: true, new: true }         // create if not found
  );

  // Send email via Nodemailer (fire-and-forget from controller)
  await mailService.sendOtpEmail(email, code);
};

exports.verifyOtp = async (email, inputCode) => {
  const record = await OTP.findOne({ email });
  if (!record) throw new Error("OTP expired. Please request a new one.");
  if (record.expiresAt < new Date()) throw new Error("OTP has expired.");
  if (record.code !== String(inputCode).trim()) throw new Error("Invalid OTP.");
  await OTP.deleteOne({ email }); // Consume the OTP (single use)
};
```

- **Cryptographically secure**: crypto.randomInt() uses the OS CSPRNG (not Math.random() which is predictable). A 6-digit OTP has 900,000 possibilities - secure for a 10-minute window.
- **MongoDB TTL index**: The OTP collection has a TTL index on expiresAt. MongoDB automatically deletes expired OTP documents - no cron job needed.
- **Upsert pattern**: findOneAndUpdate with upsert:true ensures only ONE active OTP per email. Resending OTP replaces the previous one.
- **Single use**: OTP is deleted after verification. It cannot be used twice (replay attack prevention).
- **Rate limiting**: express-rate-limit on /api/auth/resend-otp prevents OTP spam attacks.

SECTION 8

Attack Patterns & EduStack Defenses

XSS, CSRF, NoSQL injection, timing attacks, OWASP Top 10 with mitigation

8.1 Cross-Site Scripting (XSS)

XSS occurs when an attacker injects malicious JavaScript into a web page viewed by other users. The script runs in the victim's browser with access to the page's DOM, cookies (if not httpOnly), localStorage, and network requests.

XSS Type	How It Works	EduStack Mitigation
Stored XSS	Malicious script saved in DB (e.g., in a comment), rendered to all users who view it	Input sanitization on save, escape user-generated content on render, Content-Security-Policy header
Reflected XSS	Script in URL parameter, server reflects it back in HTML response	Express does not render user input in HTML (REST API returns JSON, no server-side templating)
DOM-Based XSS	Client-side JS reads URL/DOM and writes unsanitized data to innerHTML	Using textContent instead of innerHTML in frontend JS; avoid eval()

KEY CONCEPT: httpOnly cookies are XSS-resistant: Even if an attacker successfully injects script that runs document.cookie, the edustack_token cookie is NOT accessible because httpOnly flag prevents JavaScript from reading it. The cookie is still SENT with requests (the browser handles this), but scripts cannot read its value.

8.2 Cross-Site Request Forgery (CSRF)

CSRF occurs when an attacker tricks a victim's browser into making a state-changing request to a trusted site using the victim's existing session. Example: An email contains an image tag <img src="https://edustack.com/api/payments/create-order"> - the victim's browser automatically sends the request WITH their cookie.

- **SameSite=strict defense**: When EduStack sets sameSite: "strict" on the JWT cookie, browsers refuse to send the cookie on cross-origin requests. The attacker's CSRF request from their site does not include the edustack_token cookie.
- **SameSite=none in production**: EduStack uses "none" in production for Google OAuth redirect compatibility. This reduces CSRF protection, so an additional CSRF token should be implemented in production for state-changing endpoints.
- **JSON-only API**: EduStack only accepts application/json content type for state-changing endpoints. HTML form submissions (the traditional CSRF vector) do not have the Content-Type header and are rejected by express.json() middleware.
- **Helmet headers**: helmet() sets X-Frame-Options: SAMEORIGIN which prevents clickjacking (a CSRF variant using iframes).

8.3 NoSQL Injection

JavaScript / Node.js

```
// WITHOUT mongoSanitize  -  VULNERABLE to NoSQL injection:
// Attacker sends: POST /api/auth/login
// Body: { "email": { "$gt": "" }, "password": { "$gt": "" } }
// Mongoose query becomes: User.findOne({ email: { $gt: "" }, password: { $gt: "" } })
// $gt: "" matches any non-empty string  -  authentication BYPASSED!

// WITH mongoSanitize  -  PROTECTED:
app.use(mongoSanitize());
// mongoSanitize strips all keys starting with "$" from req.body and req.query
// Result: { email: "", password: "" }  -  treated as empty strings, not operators

// ADDITIONAL PROTECTION in Mongoose schemas:
// Mongoose schema casting also protects:
// email: { type: String }  -  if { $gt: "" } is passed as email,
// Mongoose tries to cast it to String, resulting in "[object Object]"
// which won't match any real email in the DB.

// BEST PRACTICE: Use parameterized queries / Mongoose typed schemas
// AND mongoSanitize as defense-in-depth.
```

8.4 Timing Attacks & timingSafeEqual

A timing attack measures the time it takes for a server to respond to determine secret information. If a string comparison exits early on first mismatch (as === does), an attacker can measure nanosecond differences in response time to determine how many characters of the secret match their guess.

JavaScript / Node.js

```
// services/razorpayService.js  -  Timing-safe comparison
const crypto = require("crypto");

const verifyPaymentSignature = (orderId, paymentId, signature) => {
  // Build the message Razorpay signed
  const message = `${orderId}|${paymentId}`;

  // Compute the expected signature using our KEY_SECRET
  const expectedSig = crypto
    .createHmac("sha256", process.env.RAZORPAY_KEY_SECRET)
    .update(message)
    .digest("hex");

  // WRONG (vulnerable to timing attack):
  // return expectedSig === signature; // Exits on first char mismatch

  // CORRECT (timing-safe  -  always takes the same time regardless of match):
  try {
    return crypto.timingSafeEqual(
      Buffer.from(expectedSig, "hex"),
      Buffer.from(signature,   "hex")
    );
  } catch {
    return false; // Length mismatch also means invalid
  }
};
```

SECTION 9

Razorpay Payment Security

HMAC-SHA256 signature verification, 2-step payment flow, fraud prevention

9.1 Razorpay 2-Step Payment Flow

FLOW: EduStack Razorpay Payment Flow

Step 1: POST /api/payments/create-order (authenticated). Server calls Razorpay API to create order with amount in paise. Saves order to DB (status: "created"). Returns orderId, amount, keyId to frontend.

↓

Step 2: Frontend opens Razorpay checkout UI with orderId + keyId. User enters card/UPI details. Payment is processed by Razorpay.

↓

Step 3: On success, Razorpay gives frontend: { razorpayOrderId, razorpayPaymentId, razorpaySignature }

↓

Step 4: POST /api/payments/verify. Server recomputes HMAC-SHA256(orderId|paymentId, KEY_SECRET) and compares with razorpaySignature using timingSafeEqual.

↓

Step 5: If valid: Update payment to "paid", set user.isPremium = true. If invalid: Set payment to "failed", return 400 (fraud attempt).

FAANG TIP: Interview Q: "Why does EduStack verify the Razorpay signature server-side instead of trusting the frontend?" The frontend is untrusted. An attacker can intercept Razorpay's callback and forge a payment confirmation. Server-side HMAC verification ensures only Razorpay (who knows the KEY_SECRET) could have generated the valid signature. This prevents fake payment injections.

SECTION 10

40 Deep Interview Q&As - Auth & Security

bcrypt, JWT, OAuth, cookies, XSS, CSRF, HMAC, OWASP - FAANG level

About This Section: These 40 questions cover authentication, security, and cryptography at the depth expected in FAANG/ Tier-1 company interviews. All answers reference EduStack's actual production security implementation.

Q1: Why should you NEVER store passwords in plain text? What is bcrypt?

Answer:

Plain-text passwords in a database expose ALL user accounts if the database is breached. bcrypt is a password hashing algorithm designed to be deliberately slow (computationally expensive), making brute-force attacks infeasible. It generates a unique random salt per password, preventing rainbow table attacks.

-> SHA-256 can hash 1 billion passwords/second on a GPU. bcrypt at cost 12 does ~100 hashes/second. 10-million-fold slowdown makes brute force impractical.

-> bcrypt embeds the salt in the hash output - bcrypt.compare() can extract it and use it for verification without a separate storage column.

-> EduStack uses bcryptjs (pure JavaScript bcrypt) with cost factor 12 - balancing security (~300ms hash time) and user experience.

Q2: What is a JWT? How does signature verification work?

Answer:

JWT (JSON Web Token) is a compact, URL-safe token with three Base64URL-encoded parts: Header (algorithm type), Payload (claims like userId and exp), and Signature (HMAC of header+payload using a secret). Any modification to header or payload invalidates the signature - tampering is detectable without contacting the server.

-> jwt.sign(payload, secret, options) creates the token. jwt.verify(token, secret) decodes and validates signature + expiry.

-> HS256: HMAC-SHA256 with a shared secret (symmetric). RS256: RSA signature with private/public key pair (asymmetric).

-> JWTs are NOT encrypted by default - the payload is Base64URL-encoded (reversible). Never store sensitive data (SSN, credit card) in JWT payload.

Q3: Why does EduStack NOT store user role in the JWT payload?

Answer:

If the role is stored in the JWT, changing a user's role (e.g., revoking admin access) has no effect until the token expires (7 days in EduStack). An attacker who obtains a token for a decommissioned admin account can continue using it with admin privileges for up to 7 days.

-> EduStack's solution: JWT only contains { id: userId }. isAuth middleware fetches the FULL user from MongoDB on every request - including current role. Role changes take effect on the very next API call.

-> Trade-off: One DB query per authenticated request. For high-traffic APIs, this can be mitigated with a short JWT expiry (15 minutes) + refresh tokens, or by caching user data in Redis.

-> NEVER store passwords, payment details, or sensitive PII in JWT payloads.

Q4: What is the difference between authentication and authorization?

Answer:

Authentication: Verifying WHO you are. "Are you really Shubham Kumar?" Verified by checking credentials (password match, valid JWT). Authorization: Verifying WHAT you are allowed to do. "Is Shubham Kumar allowed to delete this subject?" Checked by role/permissions.

-> Authentication in EduStack: isAuth middleware (JWT verification + user fetch from DB).

-> Authorization in EduStack: requireRole("admin") middleware checks req.user.role === "admin" after isAuth confirms identity.

-> Route example: router.post("/subjects", isAuth, requireRole("admin"), subjectController.create) - must be authenticated AND admin.

Q5: What is XSS? How does EduStack prevent it?

Answer:

Cross-Site Scripting (XSS) is an attack where malicious JavaScript is injected into a web page. When other users view the page, the script runs in their browser, potentially stealing cookies, localStorage data, making API requests on their behalf, or redirecting them.

-> httpOnly cookies: The edustack_token cookie cannot be read by injected scripts - document.cookie returns empty.

-> Content-Security-Policy (partially): helmet() sets some CSP-related headers. A full CSP policy would restrict which scripts can execute.

-> Input validation: express-validator validates and sanitizes req.body fields. Mongoose schema types cast and reject unexpected data formats.

Q6: What is CSRF? How does SameSite cookie prevent it?

Answer:

CSRF (Cross-Site Request Forgery) tricks a victim's browser into making an authenticated request to a trusted site from an attacker-controlled page. The browser automatically includes cookies in requests, so the victim's auth cookie rides along with the attacker's request.

-> SameSite=strict: Browser refuses to send the cookie with ANY cross-origin request (form submissions, image loads, AJAX). The CSRF request from the attacker's site never includes the auth cookie.

-> SameSite=none: EduStack uses this in production for OAuth redirect compatibility. Means CSRF via cross-origin requests IS possible - additional CSRF token protection should be added for production.

-> Content-Type check: CSRF attacks typically use HTML forms (application/x-www-form-urlencoded). EduStack only accepts application/json - rejects HTML form submissions.

Q7: Explain HMAC-SHA256. How does Razorpay use it in EduStack?

Answer:

HMAC (Hash-based Message Authentication Code) is a construction that uses a cryptographic hash function (SHA-256) combined with a secret key to produce a MAC (Message Authentication Code). It provides both integrity (message not modified) and authenticity (message came from someone with the key).

-> HMAC-SHA256 formula: HMAC = SHA256(key XOR opad || SHA256(key XOR ipad || message))

-> Razorpay signs: HMAC-SHA256("orderId|paymentId", KEY_SECRET). EduStack recomputes this. If they match, the payment is genuine.

-> crypto.timingSafeEqual() prevents timing attacks - comparison takes constant time regardless of how many characters match.

Q8: What is select: false in Mongoose? Why does EduStack use it on password?

Answer:

select: false means the field is NOT included in query results by default. You must explicitly request it with .select("+password"). This prevents the password hash from being accidentally included in API responses (e.g., when fetching user profiles).

-> Without select: false - User.findById(id) would return { name, email, password: "$2b$12$..." } - the hash leaks in every user fetch.

-> With select: false - User.findById(id) returns { name, email } - password omitted.

-> EduStack only includes password when needed for verification: User.findOne({ email }).select("+password") in the login controller.

Q9: What is OAuth 2.0 and why does EduStack use it?

Answer:

OAuth 2.0 is an authorization framework enabling third-party applications to access user accounts on external services without exposing credentials. "Sign in with Google" uses OAuth - EduStack never sees the user's Google password. Google authenticates the user and gives EduStack a profile.

-> Benefits: No password management for Google users, trusted Google verification, auto-verified email (isVerified: true for Google users).

-> Security: The authorization code (from Google's redirect) is exchanged for an access token server-to-server - the code is never exposed in the browser URL in a way that persists.

-> PKCE (Proof Key for Code Exchange) prevents code interception attacks. Passport.js handles PKCE automatically for Google OAuth.

Q10: Explain the difference between symmetric and asymmetric cryptography.

Answer:

Symmetric: Same key for encryption and decryption (e.g., AES-256). Fast but requires secure key exchange. Both parties must know the secret. Used for: bulk data encryption (TLS data phase), JWT HS256.

-> Asymmetric: Public key encrypts/verifies, private key decrypts/signs (e.g., RSA-2048, ECC). Slower but solves key distribution. Used for: TLS handshake (key exchange), JWT RS256, SSL certificates, digital signatures.

-> TLS uses both: RSA/ECDH (asymmetric) to securely exchange a session key, then AES (symmetric) for all data in that session.

-> EduStack JWT uses HS256 (symmetric) - appropriate for a single-server application. Multi-service architectures prefer RS256 so services can verify without knowing the private key.

Q11: What is a rainbow table attack? How does bcrypt's salt prevent it?

Answer:

A rainbow table is a precomputed lookup table mapping common passwords to their hashes (e.g., SHA256("password123") -> "ef92b7..."). An attacker who obtains the DB can instantly reverse common passwords without brute-forcing them.

-> bcrypt's salt prevention: bcrypt generates a random 128-bit salt per password. Even if two users have the same password, their bcrypt hashes are completely different (different salts). Rainbow tables are useless because they would need to precompute all combinations of (password, salt).

-> Attacker would need to compute 2^128 tables (one per possible salt) - computationally impossible.

-> The salt is stored IN the bcrypt hash string itself - bcrypt.compare() knows to use it.

Q12: What is a timing attack? How does crypto.timingSafeEqual prevent it?

Answer:

Regular string comparison (===) short-circuits on the first differing character. An attacker can measure nanosecond response time differences to determine how many leading characters of their guess match the correct value, enabling character-by-character brute force.

-> timingSafeEqual: Compares two Buffers in constant time - always takes the same time regardless of where the first mismatch occurs. Completely eliminates timing-based information leakage.

-> Critical for: HMAC verification (Razorpay payment sig), API key comparison, password comparison (bcrypt handles this internally).

-> Do NOT use === for security-sensitive comparisons. Always use crypto.timingSafeEqual() in Node.js.

Q13: What is the OTP model's TTL index in MongoDB? How does it work?

Answer:

A TTL (Time-To-Live) index in MongoDB automatically deletes documents after a specified time period. The OTP model has a TTL index on the expiresAt field. MongoDB's background TTL monitor runs every 60 seconds and removes expired OTP documents automatically.

-> Schema: { expiresAt: { type: Date }, code: String, email: String }

-> Index: otpSchema.index({ expiresAt: 1 }, { expireAfterSeconds: 0 }) - documents are deleted when expiresAt is in the past.

-> No manual cleanup needed - MongoDB handles OTP expiration automatically. Expired OTPs cannot be used because findOne() returns null for deleted documents.

Q14: Why is the same error message returned for wrong email AND wrong password in EduStack's login?

Answer:

Returning different messages ("Email not found" vs "Wrong password") allows an attacker to enumerate which emails are registered. They can write a script to test thousands of emails and learn which ones have accounts - useful for phishing targeted attacks.

-> EduStack returns "Invalid email or password." for BOTH cases - the attacker cannot distinguish between them.

-> Applied consistently: forgotPassword endpoint also returns the same message whether the email exists or not: "If an account with this email exists, a reset OTP has been sent."

-> This is OWASP API Security Top 10 recommendation: API2:2023 - Broken Authentication - avoid leaking information about account existence.

Q15: Explain bcrypt.genSalt() vs bcrypt.hash() - what is the difference?

Answer:

bcrypt.genSalt(rounds) generates a random cryptographic salt with the specified cost factor embedded. bcrypt.hash(password, saltOrRounds) can accept either the salt (from genSalt) or directly a number of rounds (it generates salt internally).

-> bcrypt.genSalt(12) then bcrypt.hash(password, salt): Explicit two-step - gives you the salt if you need it separately.

-> bcrypt.hash(password, 12): Generates salt internally and hashes in one call. EduStack uses this in the authController.

-> Both are equivalent in security - single-step hash is cleaner for most use cases.

Q16: What is Passport.js? How does serializeUser/deserializeUser work?

Answer:

Passport.js is authentication middleware for Express. It abstracts authentication strategies (local, Google, GitHub, etc.) into a unified API. serializeUser determines what data is stored in the session (typically just user ID). deserializeUser retrieves the full user from DB using the stored ID on subsequent requests.

-> serializeUser: passport.serializeUser((user, done) => done(null, user._id)) - stores only user._id in session (small).

-> deserializeUser: passport.deserializeUser(async (id, done) => { const user = await User.findById(id); done(null, user); }) - fetches full user from DB per request.

-> Passport is only used for Google OAuth in EduStack. Regular login uses JWT (stateless) - no session needed.

Q17: What is the difference between authorization code grant and implicit grant in OAuth?

Answer:

Authorization Code Grant (used by EduStack): Client gets an authorization code, exchanges it server-to-server for an access token. The access token is never exposed in the browser URL. Secure for web applications.

-> Implicit Grant (deprecated): The access token is returned directly in the URL fragment (e.g., #access_token=...). The token is exposed in browser history, logs, and referrer headers. Deprecated in OAuth 2.1.

-> PKCE (Proof Key for Code Exchange): Extension to Authorization Code Grant for mobile/SPA apps. Generates a code_verifier + code_challenge to prevent code interception. Passport.js supports PKCE.

-> EduStack uses Authorization Code Grant via Passport.js GoogleStrategy - the most secure OAuth grant type.

Q18: What does helmet() do? List 5 specific security headers it sets.

Answer:

Helmet is a collection of 15 middleware functions that set HTTP security headers. It protects against several well-known web vulnerabilities by controlling browser security features.

-> X-DNS-Prefetch-Control: off - Disables DNS prefetching, preventing some information leakage.

-> X-Frame-Options: SAMEORIGIN - Prevents clickjacking by refusing to display page in iframes from other origins.

-> X-Content-Type-Options: nosniff - Browser must respect Content-Type, preventing MIME-type sniffing attacks.

-> Referrer-Policy: no-referrer - Prevents referrer header from leaking URLs when navigating to external sites.

-> Strict-Transport-Security (HSTS): max-age=15552000; includeSubDomains - Forces HTTPS for 180 days.

Q19: How does express-mongo-sanitize protect against NoSQL injection?

Answer:

express-mongo-sanitize removes all keys from req.body and req.query that begin with "$" (MongoDB operators) or contain dots (.) which could be used to access nested paths. This prevents attackers from embedding MongoDB operators in user input.

-> Attack vector without protection: POST /login with { "email": { "$gt": "" } } matches ALL users (any non-empty email > "").

-> With mongoSanitize: The "$gt" key is stripped, resulting in { "email": "" } - a legitimate string query.

-> Registration order matters: app.use(mongoSanitize()) BEFORE routes ensures all request bodies are sanitized before reaching controllers.

Q20: What is the fire-and-forget pattern? When does EduStack use it?

Answer:

Fire-and-forget means initiating an async operation without awaiting its completion. The calling function returns immediately without waiting for the async operation to finish. If the operation fails, the failure is logged but does NOT affect the caller's response.

-> EduStack fires-and-forgets: mailService.sendWelcomeEmail(user.email).catch(err => console.warn(...)) - no "await". If email delivery fails, the user is still verified and logged in. Email failure is non-critical.

-> Contrast with: await otpService.saveAndSendOtp() during registration - this IS awaited because the OTP must be saved before responding.

-> Use fire-and-forget only for non-critical side effects. For business-critical operations (payment confirmation, order creation), always await and handle failures.

Q21: What is JWT token expiry? What happens when a token expires in EduStack?

Answer:

JWT tokens have an exp (expiration) claim that is a Unix timestamp. jwt.verify() checks the exp claim and throws TokenExpiredError if the current time is past the expiry. EduStack's JWT expires after 7 days (configurable via JWT_EXPIRES_IN env var).

-> When expired: isAuth catches TokenExpiredError and returns 401 "Session expired. Please log in again." The frontend should redirect to the login page.

-> No auto-refresh: EduStack does not implement refresh tokens. Users must log in again after 7 days. Refresh token implementation would use a long-lived refresh token to silently issue new short-lived access tokens.

-> JWTs cannot be invalidated before expiry (no token blacklist in EduStack). If a token is compromised, the attacker has access until expiry. Mitigation: Short expiry (15-60 minutes) + refresh tokens.

Q22: What is the principle of least privilege? How does EduStack implement it?

Answer:

Least privilege: Grant only the minimum permissions necessary for a task. Never give more access than required. An admin account compromised should not allow database root access; a user account should not allow admin operations.

-> EduStack roles: "user" - browse subjects, access resources, DSA sheet (if premium). "admin" - create/edit/delete subjects, resources, manage users. "contributor" - intermediate role.

-> requireRole("admin") middleware: Attached to admin-only routes. If req.user.role !== "admin", returns 403 Forbidden.

-> JWT payload: Only user ID stored - no role in token. Role is always freshly fetched from DB, preventing stale elevated permissions.

Q23: How does multer work in EduStack for avatar uploads? What is memoryStorage?

Answer:

Multer is a multipart/form-data middleware for Express. EduStack uses multer with memoryStorage, which stores the uploaded file in an in-memory Buffer (req.file.buffer) rather than writing to disk. The Buffer is then converted to a base64 data URI and uploaded directly to Cloudinary.

-> multer({ storage: multer.memoryStorage() }) - file never touches the filesystem, stored in RAM.

-> bufferToBase64Uri(file): Creates a "<base64String>" URI from the buffer. Cloudinary accepts this format.

-> Why not disk storage? Render.com has a read-only filesystem and ephemeral /tmp - files written to disk are lost on process restart. Memory storage + Cloudinary upload is the correct stateless approach.

Q24: What is OWASP Top 10? Name 5 items and how EduStack addresses them.

Answer:

OWASP (Open Web Application Security Project) Top 10 is a list of the most critical web application security risks.

-> A01 - Broken Access Control: EduStack uses isAuth + requireRole for all protected endpoints. Admin-only routes explicitly require admin role.

-> A02 - Cryptographic Failures: bcrypt for passwords (not MD5/SHA1), HTTPS for all production traffic (Render.com enforces HTTPS), HMAC-SHA256 for payment verification.

-> A03 - Injection: mongoSanitize prevents NoSQL injection, express-validator sanitizes inputs, Mongoose schema typing rejects operator objects.

-> A05 - Security Misconfiguration: helmet() sets secure headers. JWT_SECRET and other secrets are ONLY in environment variables, never hardcoded in source.

-> A07 - Identification and Authentication Failures: httpOnly cookie for JWT, bcrypt 12 rounds, OTP email verification, email normalization preventing case-mismatch duplicate accounts.

Q25: What is the difference between a session and a JWT?

Answer:

Session (Stateful): Server stores session data (userId, role, data). Client has a session ID cookie. Each request: server looks up session in storage (Redis, MongoDB, memory). Can be instantly invalidated (delete session from storage).

-> JWT (Stateless): All data encoded in the token itself. Server validates the signature - no storage lookup needed. Cannot be invalidated before expiry (unless using a blacklist).

-> EduStack hybrid: JWT for API requests (no DB lookup for auth, just signature verify + user fetch). MongoDB session for Google OAuth flow (Passport requires session to maintain OAuth state between redirects).

-> Performance: JWT saves a session lookup per request. Trade-off: Compromised JWT valid until expiry.

Q26: What is express-rate-limit and why is it important for auth routes?

Answer:

express-rate-limit limits the number of requests a single IP can make in a time window. Applied to auth routes, it prevents brute-force attacks (trying thousands of passwords), OTP enumeration (trying all 6-digit codes), and denial-of-service attacks.

-> Without rate limiting: An attacker can send 1 million login attempts from one IP.

-> With rate limiting: After N failures in T minutes, the IP is blocked for a period. EduStack applies rate limiting to /api/auth/login, /api/auth/register, /api/auth/resend-otp.

-> Rate limit by IP (trust proxy must be set correctly so rate limiter uses real IP, not load balancer IP).

Q27: What is environment variable security? What happens if JWT_SECRET is exposed?

Answer:

Environment variables store secrets (keys, passwords, API tokens) outside source code. They are set on the server (Render.com dashboard) and accessed via process.env. Secrets must NEVER be committed to Git.

-> JWT_SECRET exposure: Any party with JWT_SECRET can forge valid JWTs for any user, including admin accounts. Complete authentication bypass.

-> RAZORPAY_KEY_SECRET exposure: Attacker can generate valid payment signatures and fake payment verifications - bypassing payment checks.

-> GOOGLE_CLIENT_SECRET exposure: Attacker can make OAuth requests on behalf of EduStack.

-> .gitignore: .env must always be in .gitignore. EduStack has a .env.example with placeholder values for documentation without exposing actual secrets.

Q28: How does EduStack handle admin role assignment? Why is it never hardcoded?

Answer:

Admin emails are stored ONLY in environment variables (ADMIN_EMAILS=admin@example.com,admin2@example.com). During registration and Google OAuth login, EduStack checks if the user's email is in this list and assigns the admin role accordingly.

-> Why not hardcode? Source code is committed to GitHub (public repo). Hardcoding admin emails exposes them to anyone who views the code. Credentials in code violate security best practices.

-> Why not user-controlled? If role was assignable via req.body.role = "admin", any user could self-promote. EduStack only assigns role from env list OR from explicit user-provided "student"/"contributor" roles (not "admin").

-> Role persistence: After initial assignment, the role is stored in MongoDB. Subsequent logins update the role if the email is (or is no longer) in the admin list.

Q29: What is Nodemailer? How does EduStack use it for OTP emails?

Answer:

Nodemailer is a Node.js module for sending emails. EduStack configures it with Gmail SMTP (or any SMTP provider). The mailService.js creates a transporter with SMTP credentials from environment variables and provides sendOtpEmail() and sendWelcomeEmail() functions.

-> SMTP credentials: EMAIL_USER and EMAIL_PASS (Gmail app password - NOT Gmail account password) are stored in .env, never in source code.

-> OTP email template: HTML email with the 6-digit OTP code, expiry notice, and EduStack branding. Sent via transporter.sendMail().

-> Gmail App Password: Google accounts with 2FA can generate 16-character app passwords for SMTP - more secure than using the main account password.

Q30: What is Content Security Policy (CSP)? Why does EduStack disable it?

Answer:

CSP is an HTTP header that tells browsers which sources are trusted for scripts, styles, images, and other resources. It prevents XSS by blocking scripts from unexpected origins.

-> EduStack sets helmet({ contentSecurityPolicy: false }) - CSP is disabled. Reason: EduStack loads CDN resources (Google Fonts, external CSS), and the frontend HTML pages load scripts from multiple origins. A strict CSP would break these resources without careful configuration.

-> Production improvement: Configure a custom CSP that allows specific known CDN domains rather than completely disabling. Use helmet's contentSecurityPolicy configuration object.

-> Without CSP, injected scripts from other origins can execute freely. The httpOnly cookie provides some protection, but CSP adds defense-in-depth.

Q31: What is the difference between authentication tokens (JWT) and refresh tokens?

Answer:

Access Token (JWT): Short-lived (15 min - 1 hour). Sent with every API request in Authorization header or cookie. If compromised, valid only for a short time.

-> Refresh Token: Long-lived (7-30 days). Stored securely (httpOnly cookie or secure storage). Used ONLY to request new access tokens when the current one expires. NOT sent with every API request.

-> Flow: Access token expires -> Client sends refresh token to /api/auth/refresh -> Server validates refresh token -> Issues new access token.

-> EduStack does NOT implement refresh tokens (single JWT, 7 days). Production enhancement: Use short access tokens (15min) + httpOnly refresh tokens (7 days) for better security.

Q32: How does password reset work in EduStack? Walk through the flow.

Answer:

POST /api/auth/forgot-password -> Server finds user by email (same success message if not found - prevents enumeration) -> Generates OTP via otpService.saveAndSendOtp() -> Sends OTP to email.

-> POST /api/auth/verify-forgot-password -> Checks OTP is valid (not expired, code matches) -> Returns { verified: true } without resetting password yet.

-> POST /api/auth/reset-password -> Verifies OTP again (or uses verified session) -> bcrypt.hash(newPassword, 12) -> user.password = hashedPassword -> user.save().

-> Security: OTP must be verified before allowing password change. Without OTP verification, anyone who knows an email could trigger a password reset.

Q33: What is the Cloudinary integration in EduStack? How is it secure?

Answer:

EduStack uses Cloudinary for image storage (user avatars, subject thumbnails). multer memoryStorage holds the file buffer. bufferToBase64Uri() converts Buffer to ... string. cloudinary.uploader.upload() sends to Cloudinary CDN.

-> Security: Cloudinary credentials (CLOUD_NAME, API_KEY, API_SECRET) are in environment variables. Files are uploaded from the server - the client never gets Cloudinary credentials.

-> The secure_url from Cloudinary uses HTTPS. Images are served via Cloudinary's global CDN with automatic format optimization.

-> Error handling: If Cloudinary upload fails (e.g., timeout), EduStack falls back to storing the base64 data URI directly in the DB - functional but not production-ideal (large DB documents).

Q34: What is cross-origin resource sharing (CORS)? When does a browser send a preflight request?

Answer:

CORS is a browser mechanism that restricts cross-origin HTTP requests. A browser sends a preflight OPTIONS request before the actual request when: (1) Method is not GET/POST/HEAD, (2) Content-Type is not application/x-www-form-urlencoded, multipart/form-data, or text/plain, (3) Custom headers are included (like Authorization).

-> Preflight: OPTIONS /api/auth/login with Access-Control-Request-Method: POST and Access-Control-Request-Headers: Content-Type, Authorization.

-> EduStack's cors() responds to preflight with Access-Control-Allow-Origin, Access-Control-Allow-Methods, Access-Control-Allow-Headers, and Access-Control-Allow-Credentials: true.

-> credentials: true in cors() is required for the browser to include cookies in cross-origin requests. Must be paired with explicit origin (not *).

Q35: What is the difference between isPremium flag and role in EduStack's User model?

Answer:

role defines the user's PERMISSION LEVEL (admin vs regular user vs contributor). It determines what actions they can take (create subjects, delete resources, etc.). isPremium defines their SUBSCRIPTION STATUS - whether they have paid for premium DSA sheet access.

-> An admin can be non-premium (staff account that manages content but hasn't purchased premium).

-> A regular user can be premium (paid subscriber with no admin permissions).

-> Checking: requireRole("admin") checks req.user.role. Premium gate checks req.user.isPremium. Both are fetched fresh from DB on every request via isAuth.

Q36: How does EduStack prevent duplicate user registrations with different email cases?

Answer:

The User schema normalizes emails: lowercase: true in the schema, plus email.toLowerCase().trim() in the controller before any query. This ensures "User@Test.com" and "user@test.com" are treated as the same email and matched by the unique index.

-> Mongoose schema: email: { type: String, unique: true, lowercase: true, trim: true }. The lowercase: true transform runs before saving.

-> Controller normalization: const normalizedEmail = email.toLowerCase().trim() before User.findOne() and User.create().

-> Without normalization: "USER@GMAIL.COM" would create a new account even though "user@gmail.com" already exists - the MongoDB index is case-sensitive by default.

Q37: What is input validation? How does EduStack use express-validator?

Answer:

Input validation ensures request data meets expected format, type, and constraints BEFORE processing. express-validator provides a declarative validation chain using check() and body() validators that can validate, sanitize, and return structured error messages.

-> Example: body("email").isEmail().normalizeEmail() - validates email format and normalizes it.

-> body("password").isLength({ min: 6 }) - minimum length check.

-> validationResult(req) collects all validation errors. If any exist, return 400 with error details before reaching controller logic.

-> EduStack has validators/ directory with separate validation files per route for clean separation of concerns.

Q38: What is a salt round in bcrypt? What cost factor should you use?

Answer:

A "round" in bcrypt means 2^N iterations of the key expansion algorithm. Cost factor 10 = 1024 iterations, Cost factor 12 = 4096 iterations, Cost factor 14 = 16384 iterations.

-> Cost factor selection: Target 100-300ms hash time on your production hardware. Too fast (< 10 rounds) = easy to brute-force. Too slow (> 14 rounds) = login takes > 1 second = poor UX.

-> EduStack uses 12 rounds (~300ms on typical cloud VM). The OWASP recommendation is minimum cost 10, recommended 12+.

-> Cost should be increased periodically as hardware improves to maintain the same effective security level.

Q39: Explain the Google OAuth callback URL configuration in EduStack. Why is it dynamic?

Answer:

The OAuth callback URL must match exactly what is registered in Google Cloud Console. EduStack needs different URLs for local development (http://localhost:3000/auth/google/callback) and Render.com production (https://myapp.onrender.com/auth/google/callback).

-> getGoogleCallbackURL() function: Checks GOOGLE_CALLBACK_URL env var first. If it contains localhost, falls back to RENDER_EXTERNAL_URL for production. Otherwise uses the env var directly.

-> Why /auth/google/callback (not /api/auth/google/callback): Google Cloud Console has the callback registered at the root path. EduStack has a root-level route for this. All other auth endpoints are under /api/auth/.

-> Security: Google validates the callback URL exactly - an attacker cannot redirect the OAuth callback to their server because the URL must match what was pre-registered.

Q40: What security practices does EduStack use for the payment flow?

Answer:

Razorpay payment security in EduStack: (1) KEY_ID (public) sent to frontend for checkout UI - safe to expose. (2) KEY_SECRET (private) NEVER sent to frontend - only used server-side for HMAC computation. (3) Orders created server-side with exact amount - frontend cannot manipulate price. (4) HMAC-SHA256 signature verification using crypto.timingSafeEqual() - prevents tampered payment confirmations.

-> Server-side amount: PREMIUM_PRICE_PAISE = 500 is hardcoded server-side. The frontend cannot pass a different amount - the order is always created for Rs.5. A malicious frontend changing the amount displayed in the UI cannot affect the actual charge.

-> Idempotent verification: Verifying the same payment twice is safe - the second call finds the existing payment record already marked "paid" and simply updates premium status again (idempotent).

-> Payment simulation for testing: /api/payments/simulate bypasses Razorpay for test environments - grants premium without real payment. Must be disabled in production.